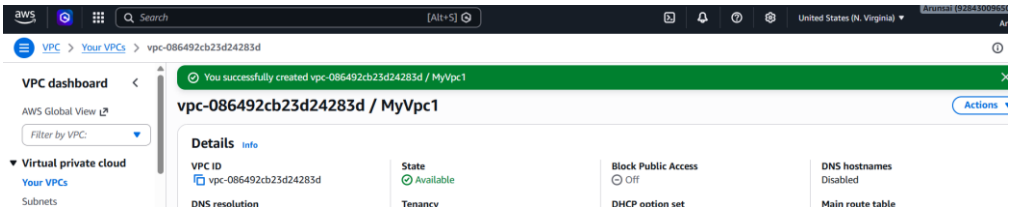
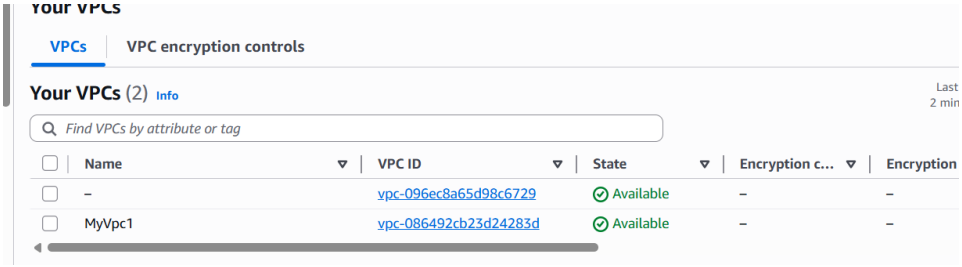
AWS AUTOSCALING

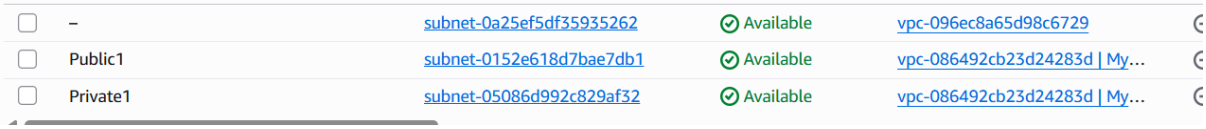
1. **Create one VPC in N. Virginia region.**
    - **Created a New VPC in N. Virgina Region**



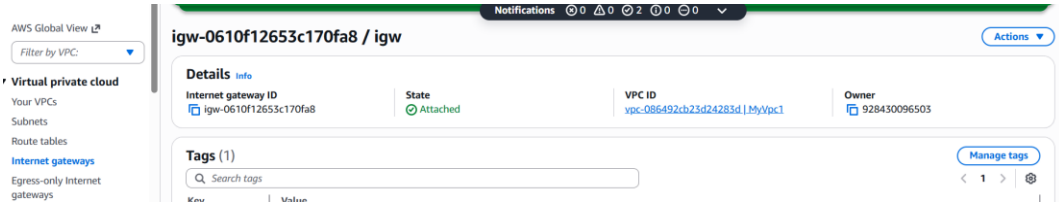
    - **MyVpc1 is new Vpc created**



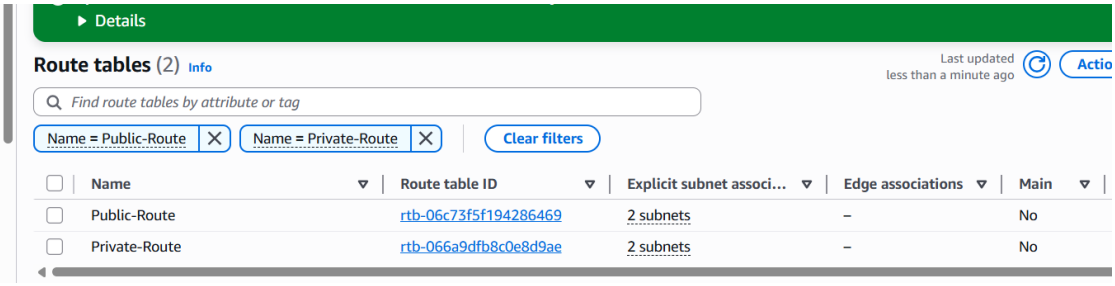
2. **Create two subnets: one public subnet and one private subnet.**
    - **Created 1 public and 1 private subnets**



3. **Attach an IGW to the VPC.**
    - **Created a new IGW and attach to myvpc1**

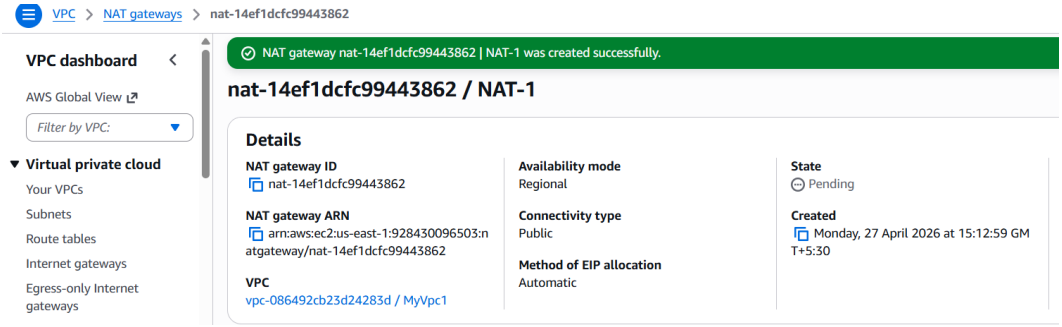


4. **Create one public route table (RT) and one private route table.**
    - **Created and 1 Public and 1 Private Route table**

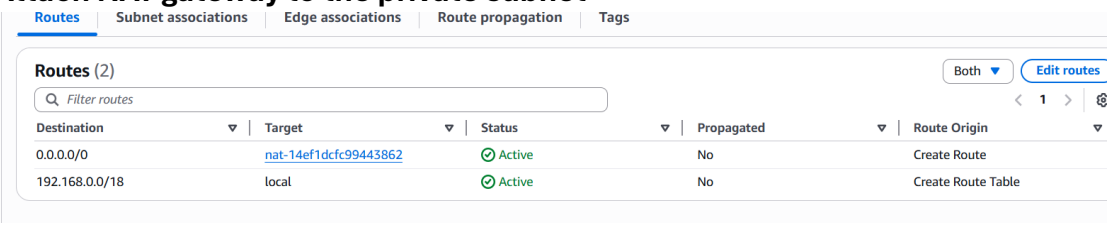


5. **Deploy a NAT gateway in the public subnet and attach the NAT gateway to the private subnet.**

   - **Create a NAT gateway in public subnet**

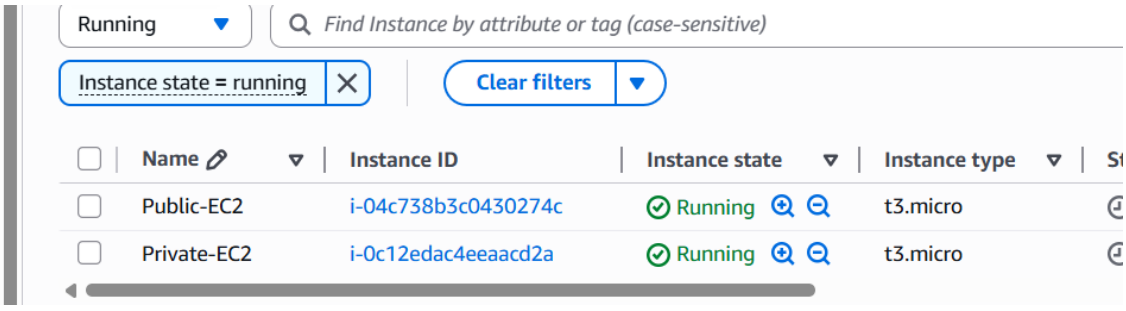


   - **Attach NAT gateway to the private subnet**



6. **Create two instances, one in the public subnet and one in the private subnet.**

   - **Created two Instances 1 is public and 1 is private subnet**

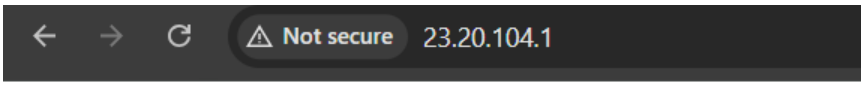


7. **Deploy Apache server on both EC2 instances with a sample index.html file.**

   - **Run the Public EC2 and install Apache server**

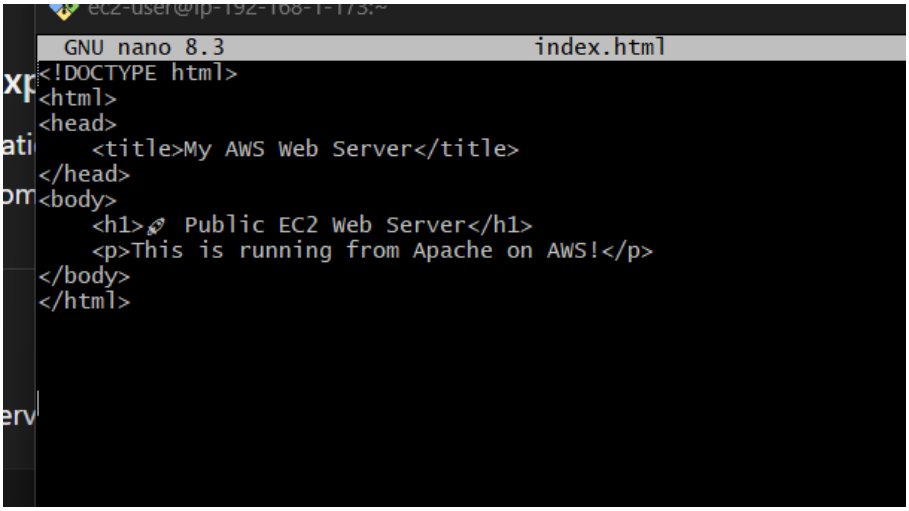
```
ogout
onnection to 192.168.3.43 closed.
ec2-user@ip-192-168-1-173 ~]$ sudo systemctl status httpd
 httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disable>
     Active: active (running) since Mon 2026-04-27 10:12:44 UTC; 32min ago
       Docs: man:httpd.service(8)
   Main PID: 26995 (httpd)
     Status: "Total requests: 3; Idle/Busy workers 100/0;Requests/sec: 0.00155; Byte>
      Tasks: 230 (limit: 1014)
     Memory: 16.9M
        CPU: 2.384s
     CGroup: /system.slice/httpd.service
             ├─26995 /usr/sbin/httpd -DFOREGROUND
             ├─26996 /usr/sbin/httpd -DFOREGROUND
             ├─26997 /usr/sbin/httpd -DFOREGROUND
```

- **Check in browser public**



It works!

- **Created index.html page and save it**



- **Now again refresh the page check the new index file is created**

Public EC2 Web Server

This is running from Apache on AWS!

- **Now private EC2 created**
- Run the public EC2 and created a vi file and paste the private pem key in that

- And given the permissions and with ssh -i pem key and user id and private id
- **Connect the private EC2 in public EC2 with pem key**

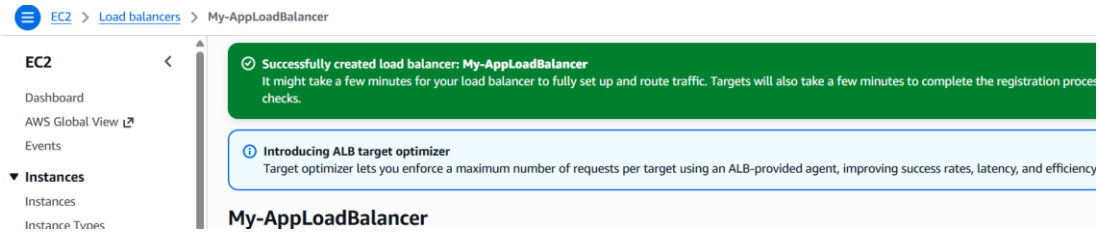
```
ec2-user@192.168.3.43: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-192-168-1-173 ~]$ vi aws.pem
[ec2-user@ip-192-168-1-173 ~]$ ssh -i aws.pem ec2-user@192.168.3.43
        #_
    ~\_  ####_        Amazon Linux 2023
   ~~  \_#####\
   ~~     \###|
   ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
    ~~       V~' '->
     ~~~         /
       ~~._.   _/
          _/ _/
        _/m/'
[ec2-user@ip-192-168-3-43 ~]$ sudo yum install httpd -y
Amazon Linux 2023 Kernel Livepatch repository       246 kB/s |  31 kB     00:00
Last metadata expiration check: 0:00:01 ago on Mon Apr 27 10:36:28 2026.
Dependencies resolved.
================================================================================
 Package                Arch       Version                    Repository      Size
================================================================================
Installing:
 httpd                  x86_64     2.4.66-1.amzn2023.0.1      amazonlinux     47 k
Installing dependencies:
```

- **Create a index file for private and save it**
- **Now check the private is running with curl url**

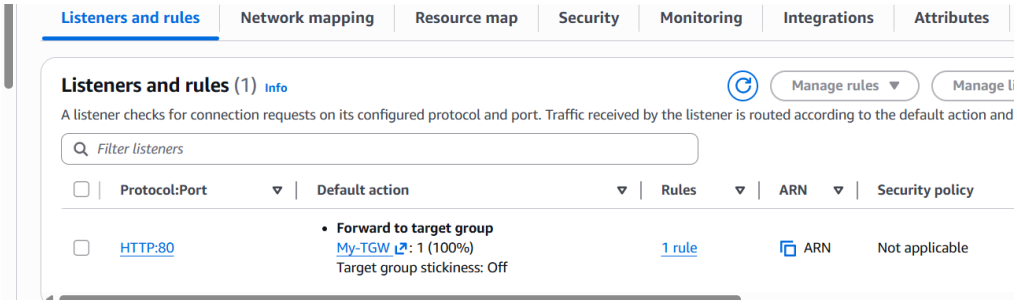
```
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.se
systemd/system/httpd.service.
[ec2-user@ip-192-168-3-43 ~]$ cd /var/www/html
[ec2-user@ip-192-168-3-43 html]$ sudo nano index.html
[ec2-user@ip-192-168-3-43 html]$ sudo systemctl restart httpd
[ec2-user@ip-192-168-3-43 html]$ ls
index.html
[ec2-user@ip-192-168-3-43 html]$
```

8. **Create one application load balancer and attach it to both EC2 instances.**

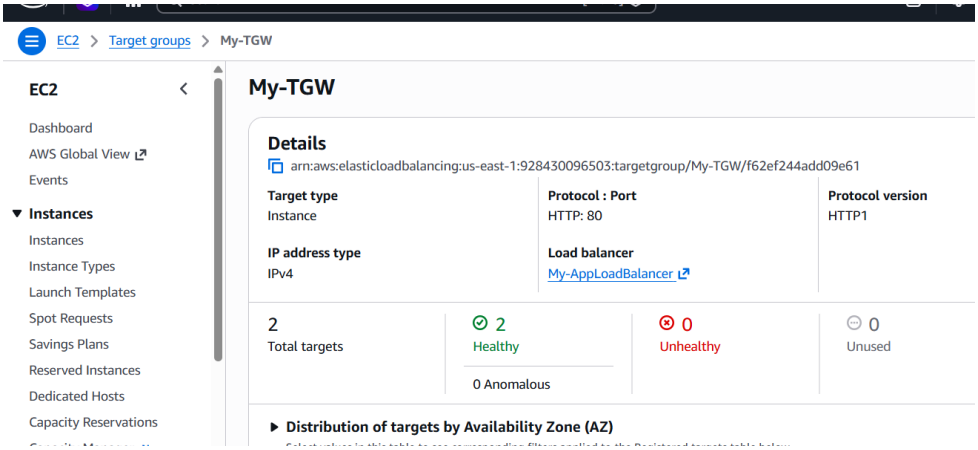
- **Created one Application load balancer**



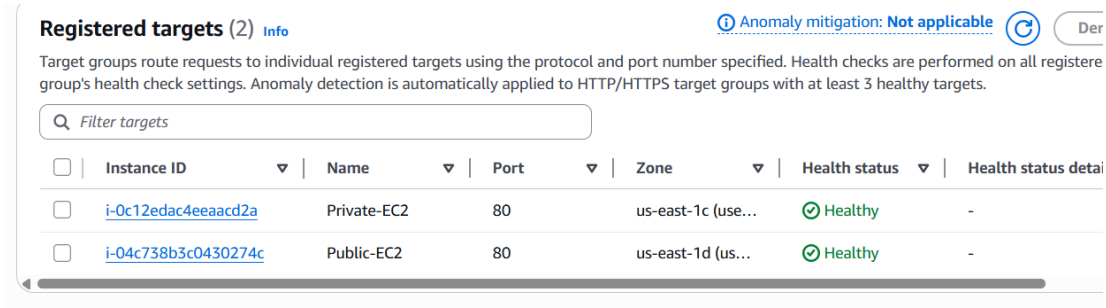
- **Given HTTP 80 and security group created and attached**



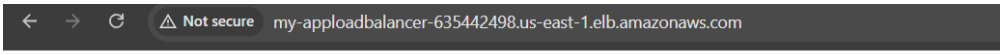
- **Created target group and to it vpc attached and also public subnets**



- **We can see the registers subnets public and private**



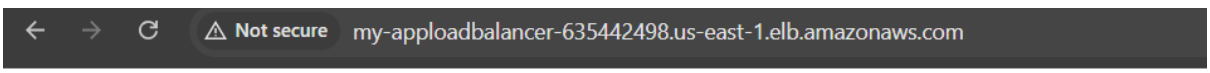
- **Now with DNS url from Application Load Balancer we can check in browser public and private index.html**



Public EC2 Web Server

This is running from Apache on AWS!

- **Now we can check private index.html which we create in earlier**

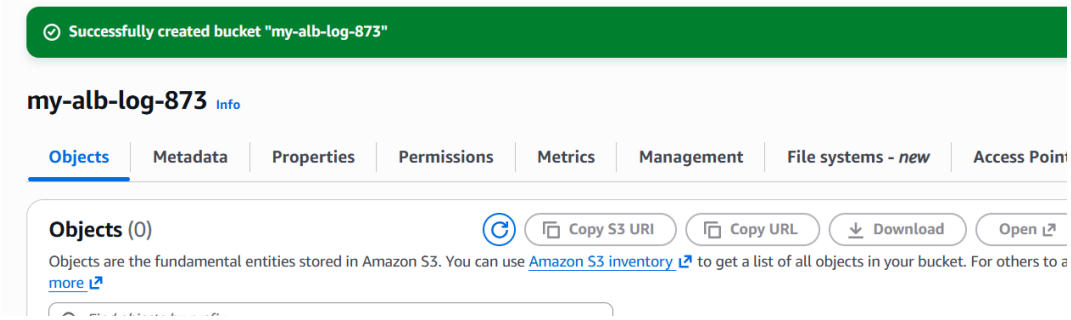


Private EC2 Server

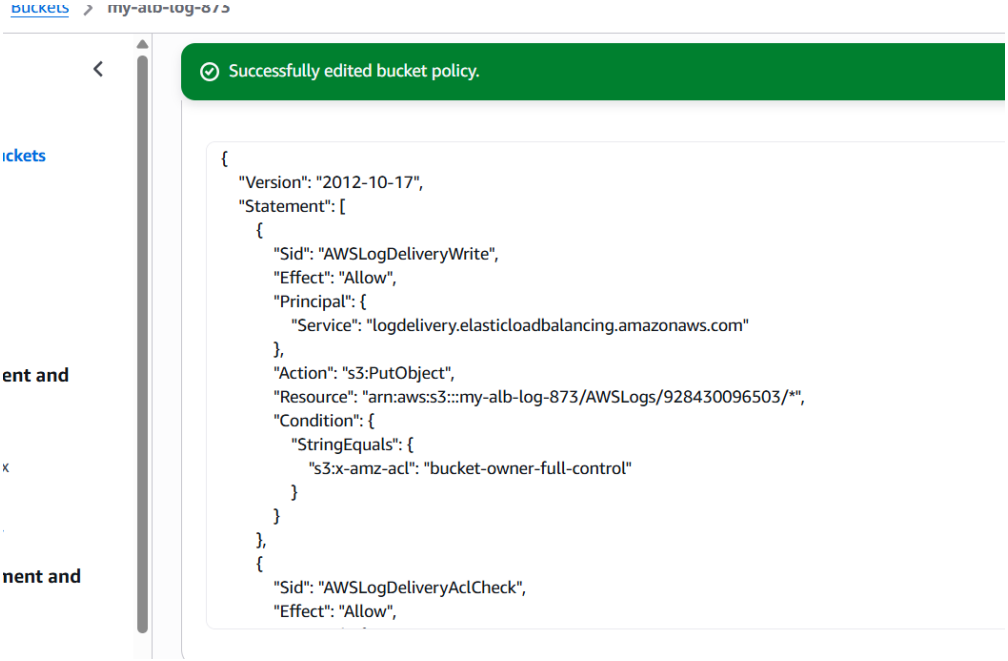
This is Private Subnet

9. **Store application load balancer logs in S3.**

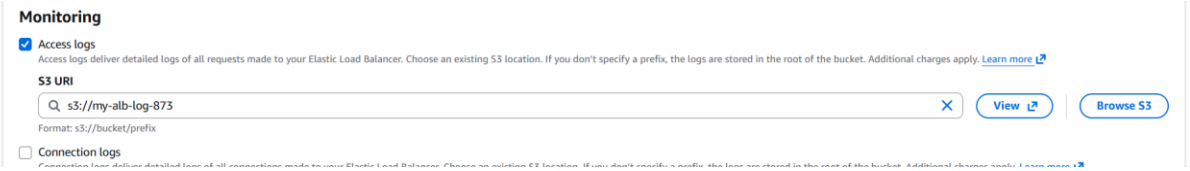
- **Created new bucket in s3 with unique and same region of ALB**



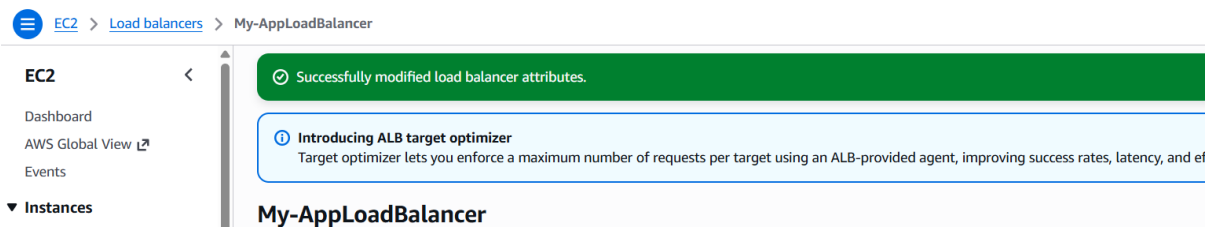
- **Given permission in bucket policy by giving my account id and bucket name**



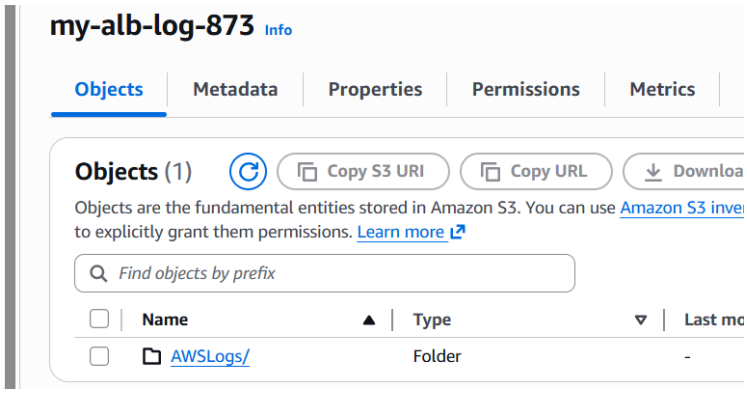
- **Now in App Load Bal open in attributes on the logs and select my new bucket**



- **ALP Attributes logs are on**

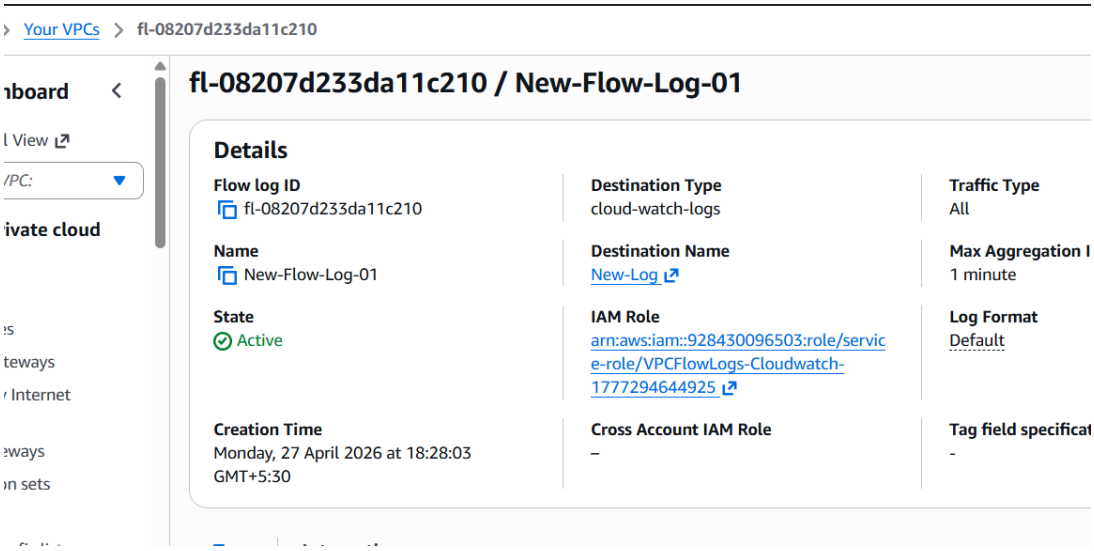


- **Now we can check in bucket folder the logs created**

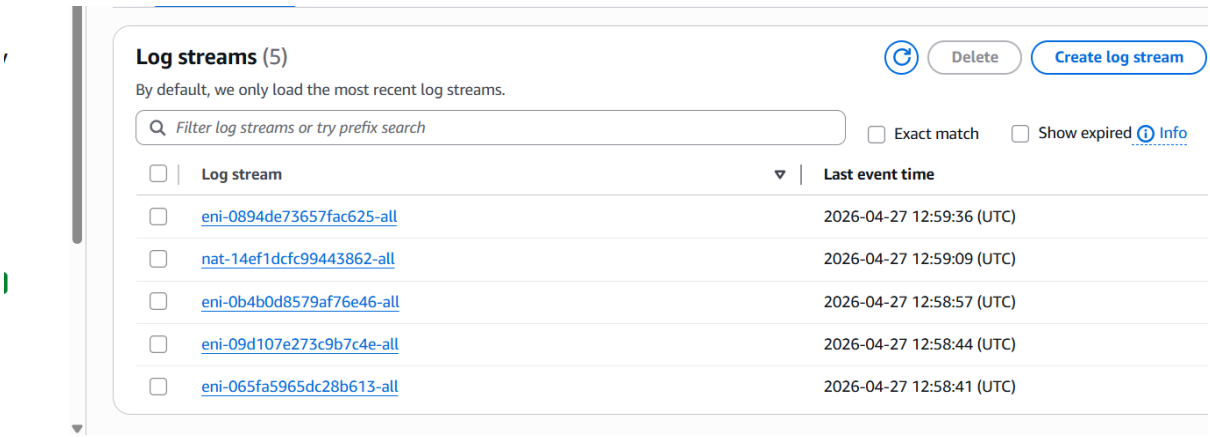


10. Store the VPC flow logs in a CloudWatch log group.

- **Create a new flow log in Vpc and send to CloudWatch from vpc**
- **Create a new IAM Role attached CloudWatch Access and save**



- **In CloudWatch check the logs files for that**

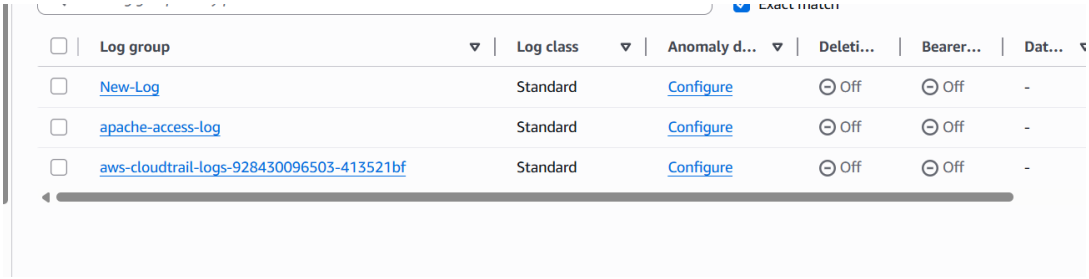


11. **Create monitoring dashboards to monitor CPU utilization and to monitor the Apache service.**

- **Run the Public EC2 and install the cloud -watch agent**

```
[ec2-user@ip-192-168-1-173 ~]$ sudo yum install amazon-cloudwatch-agent -y
Last metadata expiration check: 3:21:27 ago on Mon Apr 27 09:59:29 2026.
Dependencies resolved.
================================================================================
 Package                  Arch      Version                  Repository      Size
================================================================================
Installing:
 amazon-cloudwatch-agent  x86_64    1.300064.2-1.amzn2023    amazonlinux     68 M

Transaction Summary
================================================================================
Install  1 Package

Total download size: 68 M
Installed size: 228 M
Downloading Packages:
amazon-cloudwatch-agent-1.300064.2-1.amzn2023.x86_64  69 MB/s |  68 MB     00:00
--------------------------------------------------------------------------------
Total                                                 65 MB/s |  68 MB     00:01
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing        :                                                        1/1
  Running scriptlet: amazon-cloudwatch-agent-1.300064.2-1.amzn2023.x86_64   1/1
```

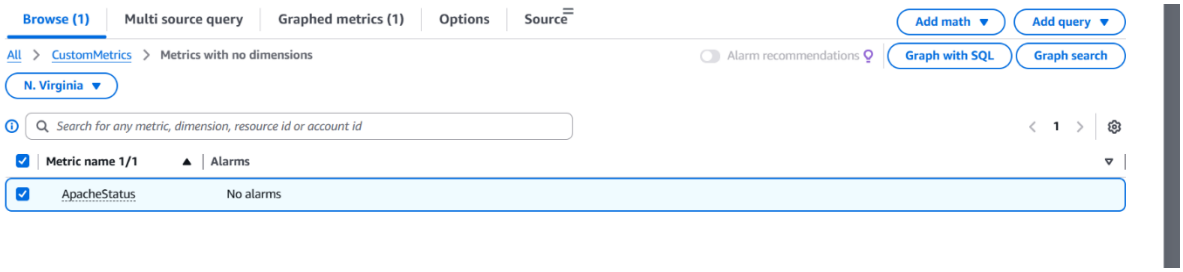
- **We can get the apache log files in CloudWatch**



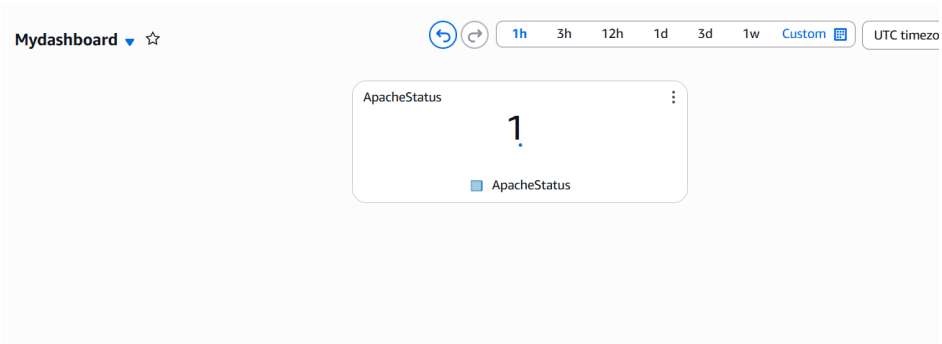
- **Create a file of apache server and save if it runs 1 and fail 0**
- **Give access permissions and also other permissions to**

```
ec2-user@ip-192-168-1-173 ~]$ sudo systemctl restart amazon-cloudwatch-agent
ec2-user@ip-192-168-1-173 ~]$ nano apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ chmod +x apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ ./apache-check.sh

ec2-user@ip-192-168-1-173 ~]$ aws configure
WS Access Key ID [None]: AKIA5QKV7AB3SMJEO2X6
WS Secret Access Key [None]: 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
efault region name [None]: us-east-1
efault output format [None]: json
ec2-user@ip-192-168-1-173 ~]$ nano apache-check.sh
ec2-user@ip-192-168-1-173 ~]$ ./apache-check.sh

ec2-user@ip-192-168-1-173 ~]$ ls
pache-check.sh  aws.pem
ec2-user@ip-192-168-1-173 ~]$ VALUE=$(./tomcat-check.sh)
bash: ./tomcat-check.sh: No such file or directory
ec2-user@ip-192-168-1-173 ~]$ VALUE=$(./apache-check.sh)
ec2-user@ip-192-168-1-173 ~]$ aws cloudwatch put-metric-data \
 --metric-name ApacheStatus \
 --namespace CustomMetrics \
 --value $VALUE
ec2-user@ip-192-168-1-173 ~]$ ^C
ec2-user@ip-192-168-1-173 ~]$
```

- **Create a new dashboard a select custom metrics and select apache status**

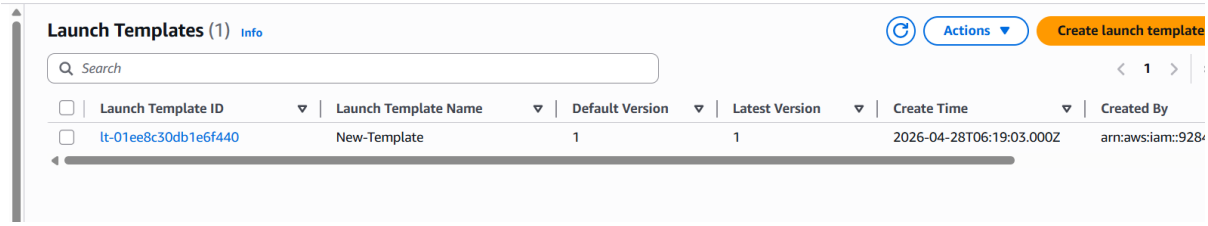


- **In dashboard we can see if it runs we get 1 else if it fails 0**

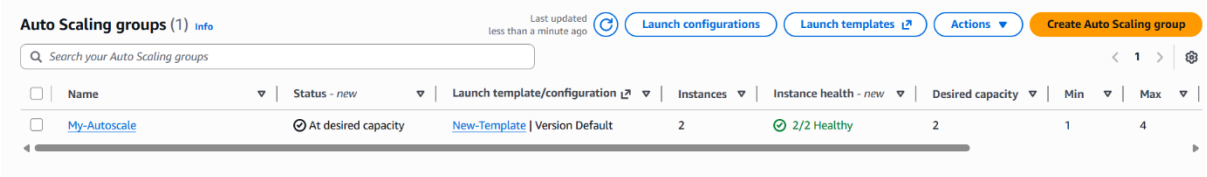


12. If CPU utilization is more than 70%, then it should trigger auto scaling and launch new instance.

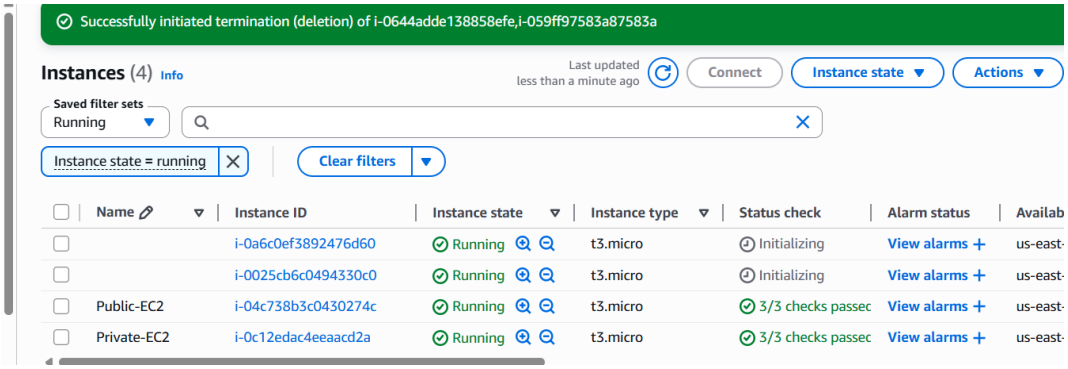
- **Created new Launch template**



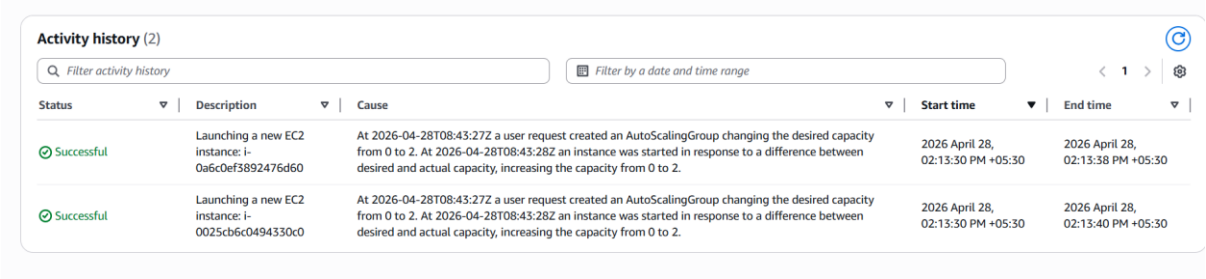
- **Created new Auto scaling group**



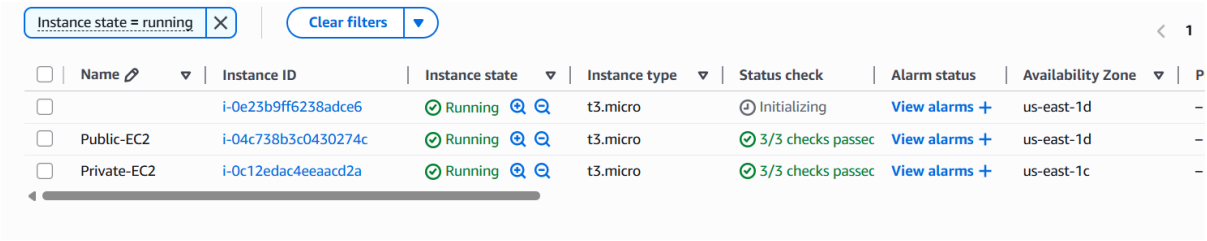
- **Two new desired instances created default after giving desired -2, mini-1, max-4**



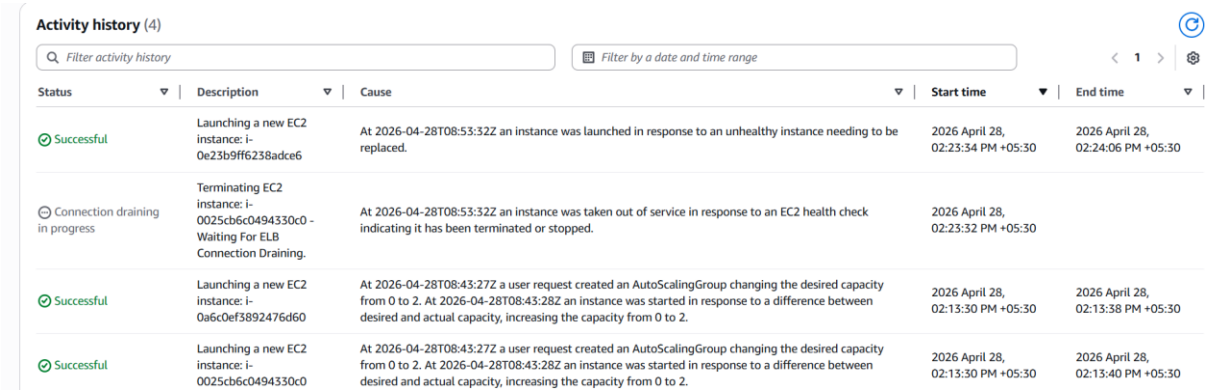
- **In Auto scaling group by selecting open it and in that activity we can see**



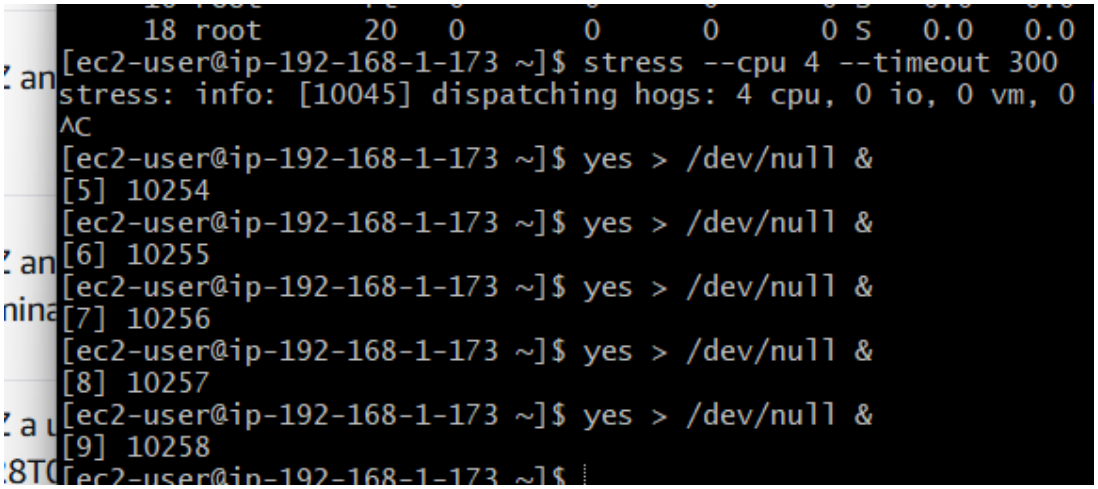
- **After terminated the default ec2 and we can see new instance created in instance**



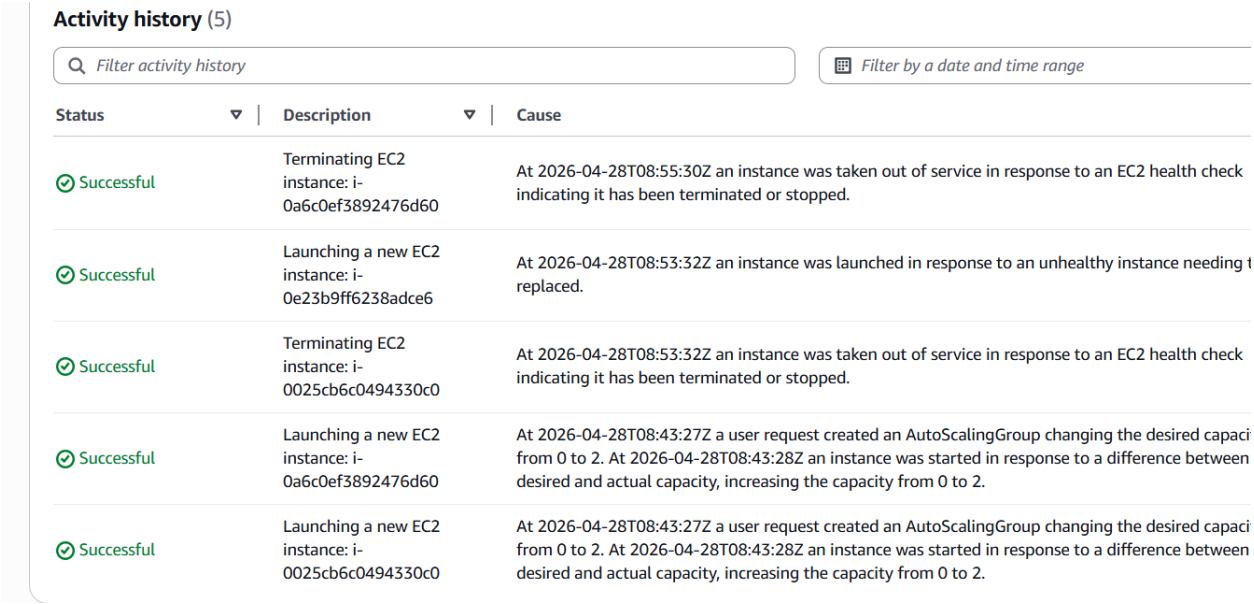
- **In Auto scale group also in active we can see new update successful one**



- **After giving more load on CPU utilization, we can see new instances created**



- **We can see in Auto Scaling Group**



Activity history (5)

Filter activity history | Filter by a date and time range

Status	Description	Cause
Successful	Terminating EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:55:30Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.
Successful	Launching a new EC2 instance: i-0e23b9ff6238adce6	At 2026-04-28T08:53:32Z an instance was launched in response to an unhealthy instance needing t replaced.
Successful	Terminating EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:53:32Z an instance was taken out of service in response to an EC2 health check indicating it has been terminated or stopped.
Successful	Launching a new EC2 instance: i-0a6c0ef3892476d60	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capaci from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.
Successful	Launching a new EC2 instance: i-0025cb6c0494330c0	At 2026-04-28T08:43:27Z a user request created an AutoScalingGroup changing the desired capaci from 0 to 2. At 2026-04-28T08:43:28Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 2.